Lab 5: Sliding-Window ARQ: Go-Back-N and Selective Repeat

Computer Networks · Week 5 · Data Link Layer

In this lab you implement two sliding-window ARQ protocols, Go-Back-N and Selective Repeat, over the same three components you built before: a sender, a receiver, and a transmission medium that carries frames between them. The medium is the scripted channel from Lab 3, now able to drop frames and acknowledgements as well, so that retransmission logic has something to recover from. You then run both protocols over identical loss schedules and measure how differently they behave.

As before, the sender and receiver never communicate directly. Data frames travel sender to medium to receiver, and acknowledgements travel receiver to medium to sender. A frame or an acknowledgement is lost only if the medium is scripted to drop it.

You will:

- implement Go-Back-N: cumulative acknowledgements, a receiver that discards out-of-order frames, and a sender that resends the whole outstanding window on timeout;
- implement Selective Repeat: individual acknowledgements, a receiver that buffers out-of-order frames, and a sender that resends only the missing frame;
- run both protocols over identical loss schedules and measure how many transmissions each one needs;
- analyse the efficiency difference and derive why Selective Repeat needs a larger sequence-number space than Go-Back-N.

Conventions and constants

- Payloads are a fixed list; each is given a sequence number 0, 1, 2, and so on up to N minus 1.
- A data frame is a pair (seq, payload).
- An acknowledgement is a single sequence number. Go-Back-N uses a **cumulative** acknowledgement: an ACK for a means every sequence number up to and including a has been received in order. Selective Repeat uses an **individual** acknowledgement: an ACK for a means sequence number a specifically has been received.
- W is the window size, given per experiment.
- Sequence numbers are plain integers 0 to N minus 1 in this lab.

The transmission medium

The medium is the scripted channel from Lab 3, extended to drop frames. It is driven by a drop set:

drops is a set of (round, kind, seq) entries, where *kind* is either data or ack.

- A data frame with sequence number s sent in round r is lost if (r, data, s) is in the drop set.
- An acknowledgement for a sent in round r is lost if (r, ack, a) is in the drop set.
- Everything not named in the drop set is delivered. An empty drop set is a perfect channel.

The drop set is fixed for each experiment, so every run is deterministic and reproducible, exactly as in Lab 3.

The simulation model (round-based and deterministic)

Time advances one round at a time. In each round, in this order:

1. **Send.** For each sequence number in the sender window (from base to base + W - 1, limited to valid numbers), transmit it if it has never been sent, or if it was sent in an earlier round and is still unacknowledged. The second case is a timeout: one round without acknowledgement counts as expired. Count every transmission.
2. **Data channel.** Each sent frame is delivered unless its (round, data, seq) is in the drop set.
3. **Receiver.** Process the delivered frames in ascending sequence order.
    - Go-Back-N: accept a frame only if its sequence number equals the expected number; deliver it and advance the expected number. Discard anything out of order. If at least one frame arrived, send one cumulative acknowledgement equal to the expected number minus one.
    - Selective Repeat: accept a frame whose sequence number lies in the receive window; record it and send an individual acknowledgement for it. Re-acknowledge a frame whose number is below the window base (already delivered). Then deliver the contiguous run starting at the receive base, advancing the base.
4. **Acknowledgement channel.** Each acknowledgement is delivered unless its (round, ack, seq) is in the drop set.
5. **Sender processes acknowledgements.**
    - Go-Back-N: a cumulative acknowledgement a advances base to a + 1.
    - Selective Repeat: an individual acknowledgement a marks sequence number a acknowledged, then base advances over the contiguous run of acknowledged numbers.

6. **Timeout carry-over.** Any frame still unacknowledged at the end of the round it was sent is retransmitted in the next round.

The run ends when base reaches N, meaning every frame has been acknowledged. Report three things: the total number of data transmissions, the number of rounds, and the receiver's delivery order.

Test vectors (your output must match these exactly)

Scenario	N	W	Dropped in transit	Go-Back-N	Selective Repeat
S0 clean channel	6	4	nothing	6 transmissions, 2 rounds	6 transmissions, 2 rounds
S1 one data frame lost	6	4	data seq 2 in round 1	8 transmissions, 2 rounds	7 transmissions, 2 rounds
S2 first frame of a full window lost	8	8	data seq 0 in round 1	16 transmissions, 2 rounds	9 transmissions, 2 rounds
S3 one acknowledgement lost	4	4	ack 3 in round 1	8 transmissions, 2 rounds	5 transmissions, 2 rounds

In every scenario the delivery order is the payloads in sequence, 0 upward. The transmission counts are what differ.

Per-round transmit log for S2, which both protocols must reproduce:

- Go-Back-N. Round 1: send 0 to 7, frames 1 to 7 arrive (0 lost), no acknowledgement, base stays 0. Round 2: send 0 to 7 again, all arrive, cumulative ACK 7, base becomes 8.
- Selective Repeat. Round 1: send 0 to 7, frames 1 to 7 arrive and are buffered, individual ACKs 1 to 7, base stays 0. Round 2: send only 0, it arrives, ACK 0, base becomes 8.

The contrast in round 2 is the whole point: Go-Back-N resends all eight frames, Selective Repeat resends one.

Part A: Go-Back-N

What you do: implement the Go-Back-N sender and receiver over the medium, following the simulation model, and reproduce the Go-Back-N column of the test-vector table.

Background: Go-Back-N allows up to W unacknowledged frames in flight. The receiver is deliberately simple: it keeps no buffer, accepts frames strictly in order, and

discards anything out of order. It acknowledges cumulatively, naming the highest in-order frame it has received. When the sender's oldest unacknowledged frame times out, the sender goes back and resends that frame and every frame sent after it, because the simple receiver has thrown those away.

Your task:

1. Implement the Go-Back-N sender: a window of size W, cumulative acknowledgement handling, and go-back retransmission of the whole outstanding window on timeout.
2. Implement the Go-Back-N receiver: in-order acceptance only, discard out of order, cumulative acknowledgement.
3. Run scenarios S0, S1, S2, and S3, and confirm the transmission counts, round counts, and delivery order match the table.
4. For S2, confirm your per-round transmit log matches the one given above.

Part B: Selective Repeat

What you do: implement the Selective Repeat sender and receiver over the medium, following the simulation model, and reproduce the Selective Repeat column of the test-vector table.

Background: Selective Repeat also allows up to W unacknowledged frames, but the receiver keeps a buffer. It accepts any frame that falls within its receive window, even out of order, buffers it, and acknowledges it individually. It delivers frames to the layer above only once the contiguous run from its window base is complete. The sender therefore retransmits only the specific frames that were not acknowledged, not the whole window.

Your task:

1. Implement the Selective Repeat sender: a window of size W, individual acknowledgement handling, and retransmission of only the timed-out frames.
2. Implement the Selective Repeat receiver: windowed acceptance with buffering, individual acknowledgement, and in-order delivery of the buffered run.
3. Run scenarios S0, S1, S2, and S3, and confirm the transmission counts, round counts, and delivery order match the table.
4. For S2, confirm your per-round transmit log matches the one given above.

Part C: Comparison

What you do: run both protocols on the same loss schedules and account for the difference in transmissions.

Your task:

1. For S1 and for S2, record the total transmissions for each protocol and list exactly which sequence numbers each protocol resent.
2. Explain, in two or three sentences, why Go-Back-N resends frames that had already arrived safely, and why Selective Repeat does not.
3. State the condition under which the two protocols behave identically (that is, when Go-Back-N wastes nothing).

Part D: Analysis

What you do: quantify the efficiency gap and derive the sequence-number requirement of each protocol.

Your task:

1. Define retransmission overhead as total transmissions divided by number of payloads. Compute it for both protocols in S1 and S2, and state how the gap changes as the window grows.
2. Explain why the difference between the two protocols widens with a larger window and shrinks toward zero as the loss rate approaches zero.
3. Derive the sequence-number-space requirement. Show, with a small wraparound example, why Selective Repeat needs at least 2W distinct sequence numbers to avoid ambiguity, while Go-Back-N needs only W + 1. This part is written reasoning; it does not require code, because the lab uses plain integer sequence numbers.

What to submit

- Your source file (C++ or Python) implementing the sender, the receiver, and the medium for both protocols.
- Program output showing that both protocols reproduce the full test-vector table, including the S2 per-round log.
- Part C: the per-protocol list of resent sequence numbers for S1 and S2, and the explanation.
- Part D: the computed overheads, and the written derivation of the sequence-number requirement.

Appendix: the interface each program must implement

Implement these operations. Names and types are illustrative; only the reported counts and delivery order are graded.

- **Medium:** a drop set of (round, kind, seq) entries; a frame or acknowledgement is delivered unless it is named in the set.
- **Go-Back-N:** run_gbn(payloads, W, drops) -> (transmissions, rounds, delivery_order).
- **Selective Repeat:** run_sr(payloads, W, drops) -> (transmissions, rounds, delivery_order).

Each run executes the round-based model to completion and returns the three reported quantities. The sender and receiver state (window base, expected or receive-window base, per-frame status) lives inside these functions.